



Language Modeling With N Gram Language Models

CS 584

Schaefer School of Engineering

Justo Karell



Overview

- 1 Recap: From text to tokens
- 2 What is an n-gram?
- 3 **What is a language model?**
- 4 **Unigram probabilities: $P(w)$**
- 5 **Bigram probabilities: $P(w_2—w_1)$**
- 6 **Trigram probabilities: $P(w_3—w_1, w_2)$**
- 7 The sparsity problem
- 8 Applications: Prediction and generation

Recap: Tokenization

We learned three tokenization methods:

- Whitespace: `text.split()`
- Regex: Pattern-based splitting
- BPE: Data-driven subword units

Result: Corpus as list of tokens

Input: `str` → Output: `List[str]`

We Have Tokens: Now What?

After tokenization:

```
tokens = ["the", "cat", "sat", "on", "the", "mat"]
```

What can we do?

- Count frequencies (we did this)
- Compute statistics (N, —V—, hapax)
- But can we *predict* what comes next?

Question

What is a language model?
How do we predict the next word?

Definition: N-Gram

An **n-gram** is a contiguous sequence of n tokens.

From "the cat sat on the mat":

- Unigrams (1-grams): "the", "cat", "sat", "on", "mat"
- Bigrams (2-grams): "the cat", "cat sat", "sat on"
- Trigrams (3-grams): "the cat sat", "cat sat on"

N-grams in Python

An n-gram is a tuple of consecutive tokens:

```
tokens = ["the", "cat", "sat", "on", "the", "mat"]

# Extract bigrams
bigrams = []
for i in range(len(tokens) - 1):
    bigram = (tokens[i], tokens[i+1])
    bigrams.append(bigram)

print(bigrams)
# [('the', 'cat'), ('cat', 'sat'), ('sat', 'on'),
#  ('on', 'the'), ('the', 'mat')]
```

N-gram = tuple of strings

Extracting N-Grams

```
def get_ngrams(tokens, n):  
    """Extract all n-grams from token list."""  
    ngrams = []  
    for i in range(len(tokens) - n + 1):  
        ngram = tuple(tokens[i:i+n])  
        ngrams.append(ngram)  
    return ngrams  
  
# Unigrams  
print(get_ngrams(tokens, 1))  
# [('the',), ('cat',), ('sat',), ...]  
  
# Trigrams  
print(get_ngrams(tokens, 3))  
# [('the', 'cat', 'sat'), ('cat', 'sat', 'on'), ...]
```


Definition: Language Model

A **language model** assigns probabilities to sequences of tokens.

For each context, defines $P(\text{next word} \mid \text{context})$

Built from counting n-grams in a corpus

The model IS the n-gram counts + probabilities

Tiny Example Corpus

Let's build the simplest possible model.

Corpus:

"the cat sat"

"the dog sat"

Tokens: [the, cat, sat, the, dog, sat]

We'll count bigrams and compute probabilities.

Step 1: Count Bigrams

```
tokens = ["the", "cat", "sat", "the", "dog", "sat"]

bigram_counts = {}
for i in range(len(tokens) - 1):
    bigram = (tokens[i], tokens[i+1])
    bigram_counts[bigram] = bigram_counts.get(bigram, 0) + 1

print(bigram_counts)
# (('the', 'cat'): 1, ('cat', 'sat'): 1,
# ('sat', 'the'): 1, ('the', 'dog'): 1,
# ('dog', 'sat'): 1}
```

Step 2: Visualize Counts

Bigram count table:

w1 \ w2	cat	dog	sat	the
the	1	1	0	0
cat	0	0	1	0
dog	0	0	1	0
sat	0	0	0	1

Row = first word, Column = second word
"the" followed by "cat" happened 1 time

Step 3: Normalize to Probabilities

For each first word, normalize counts:

"the" appears 2 times total

- "the" → "cat": $1/2 = 0.5$
- "the" → "dog": $1/2 = 0.5$

"cat" appears 1 time total

- "cat" → "sat": $1/1 = 1.0$

"dog" appears 1 time total

- "dog" → "sat": $1/1 = 1.0$

Step 4: Organize Probabilities

Probability table:

P(w2—w1)	cat	dog	sat	the
the	0.5	0.5	0.0	0.0
cat	0.0	0.0	1.0	0.0
dog	0.0	0.0	1.0	0.0
sat	0.0	0.0	0.0	1.0

This table IS the language model!
Each row sums to 1.0 (probability distribution)

Using Our Model

Question: What word follows "the"?

Look at the model:

- $P(\text{cat} \text{ — the}) = 0.5$
- $P(\text{dog} \text{ — the}) = 0.5$
- $P(\text{sat} \text{ — the}) = 0.0$

Answer: "cat" and "dog" are equally likely!

The model captures what we saw in training.

Unigram Language Model

Simplest model: Each word is independent

Probability of a sequence:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

Example:

$$P(\text{the cat sat}) = P(\text{the}) \cdot P(\text{cat}) \cdot P(\text{sat})$$

Maximum Likelihood Estimation (MLE)

How do we estimate $P(w)$?

Formula:

$$P(w) = \frac{\text{count}(w)}{N}$$

where N = total tokens in corpus

Just the frequency!

Example: Unigram Probabilities (setup)

Let's compute unigram probabilities step-by-step.

Corpus: "the cat sat on the mat"

Tokens: [the, cat, sat, on, the, mat]

Goal: Compute $P(w)$ for each word

Step 1: Count Each Token

Count all tokens:

Word	Count
the	2
cat	1
sat	1
on	1
mat	1

Total tokens: $N = 6$

Step 2: Apply MLE

For each word: $P(w) = \text{count}(w) / N$

$$P(\text{the}) = \frac{2}{6} = 0.333$$

$$P(\text{cat}) = \frac{1}{6} = 0.167$$

$$P(\text{sat}) = \frac{1}{6} = 0.167$$

$$P(\text{on}) = \frac{1}{6} = 0.167$$

$$P(\text{mat}) = \frac{1}{6} = 0.167$$

Step 3: Verify Probabilities Sum to 1

Check: Do all probabilities sum to 1.0?

$$\begin{aligned} &P(\text{the}) + P(\text{cat}) + P(\text{sat}) + P(\text{on}) + P(\text{mat}) \\ &= 0.333 + 0.167 + 0.167 + 0.167 + 0.167 \\ &= 1.001 \approx 1.0 \text{ (rounding)} \end{aligned}$$

Yes! This is a valid probability distribution.

Step 4: Compute Sequence Probability

Use the unigram model to compute $P(\text{sentence})$:

$$\begin{aligned} P(\text{the cat sat}) &= P(\text{the}) \cdot P(\text{cat}) \cdot P(\text{sat}) \\ &= 0.333 \times 0.167 \times 0.167 \\ &= 0.0093 \end{aligned}$$

Unigram Model in Python

```
from collections import Counter

tokens = ["the", "cat", "sat", "on", "the", "mat"]
counts = Counter(tokens)
N = len(tokens)

# Compute probabilities
unigram_probs = {}
for word, count in counts.items():
    unigram_probs[word] = count / N

print(unigram_probs)
# {'the': 0.333, 'cat': 0.167, 'sat': 0.167,
#  'on': 0.167, 'mat': 0.167}
```

Computing Sequence Probabilities

```
def unigram_prob(sentence, probs):  
    """Compute  $P(\text{sentence})$  under unigram model."""  
    prob = 1.0  
    for word in sentence:  
        prob *= probs.get(word, 0)  
    return prob  
  
sent = ["the", "cat", "sat"]  
p = unigram_prob(sent, unigram_probs)  
print(f" $P(\text{the cat sat}) = {p:.4f}$ ")  
#  $P(\text{the cat sat}) = 0.0093$ 
```


Unigram Model Problem

What's wrong with unigrams?

No context! Every word is independent.

- $P(\text{the cat sat}) = P(\text{sat cat the})$
- Can't capture word order
- Can't capture dependencies

Example generation: "the the the cat mat on sat"
Just sampling words by frequency!

Bigram Language Model

One step better: Condition on previous word

Probability of a sequence:

$$P(w_1, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$$

Example:

$$P(\text{the cat sat}) \approx P(\text{the}) \cdot P(\text{cat}|\text{the}) \cdot P(\text{sat}|\text{cat})$$

MLE for Bigrams

Formula:

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$$

In words:

- Count how often w_1 is followed by w_2
- Divide by how often w_1 appears

Example: Bigram Probabilities (Setup)

Let's compute bigram probabilities step-by-step.

Corpus: "I like cats. I like dogs."

Tokens: [I, like, cats, I, like, dogs]

Goal: Compute $P(w_2 \mid w_1)$ for all bigrams

Step 1: Count ALL Bigrams

Extract and count bigrams:

Bigram	Count
(I, like)	2
(like, cats)	1
(cats, I)	1
(like, dogs)	1

Total: 5 bigrams (from 6 tokens)

Step 2: Count First Words (Unigrams)

For denominators, count each first word:

Word	Count
I	2
like	2
cats	1
dogs	0

Note: "dogs" never starts a bigram (appears at end)

Step 3: Apply Bigram MLE Formula

For each bigram: $P(w_2|w_1) = \text{count}(w_1, w_2) / \text{count}(w_1)$

$$P(\text{like}|I) = \frac{\text{count}(I, \text{like})}{\text{count}(I)} = \frac{2}{2} = 1.0$$

$$P(\text{cats}|\text{like}) = \frac{\text{count}(\text{like}, \text{cats})}{\text{count}(\text{like})} = \frac{1}{2} = 0.5$$

$$P(\text{dogs}|\text{like}) = \frac{\text{count}(\text{like}, \text{dogs})}{\text{count}(\text{like})} = \frac{1}{2} = 0.5$$

$$P(I|\text{cats}) = \frac{\text{count}(\text{cats}, I)}{\text{count}(\text{cats})} = \frac{1}{1} = 1.0$$

Step 4: Verify each Distribution Sums to 1

For each first word, probabilities must sum to 1:

Context "I":

$$P(\text{like}|\text{I}) = 1.0 \quad \checkmark$$

Context "like":

$$P(\text{cats}|\text{like}) + P(\text{dogs}|\text{like}) = 0.5 + 0.5 = 1.0 \quad \checkmark$$

Context "cats":

$$P(\text{I}|\text{cats}) = 1.0 \quad \checkmark$$



Step 5: Complete Bigram Model

The bigram language model:

P(w2—w1)	I	like	cats	dogs
I	0.0	1.0	0.0	0.0
like	0.0	0.0	0.5	0.5
cats	1.0	0.0	0.0	0.0
dogs	0.0	0.0	0.0	0.0

This table IS the language model!

Step 6: Use Model to Compute P(sentence)

Sentence: "I like cats"

$$\begin{aligned}P(\text{I like cats}) &= P(\text{I}) \cdot P(\text{like}|\text{I}) \cdot P(\text{cats}|\text{like}) \\&= P(\text{I}) \cdot 1.0 \cdot 0.5 \\&= P(\text{I}) \cdot 0.5\end{aligned}$$

Need $P(\text{I})$ from unigram model: $P(\text{I}) = 2/6 = 0.333$

$$P(\text{I like cats}) = 0.333 \times 0.5 = 0.167$$

Bigram Model in Python

```
# Count bigrams
bigram_counts = {}
for i in range(len(tokens) - 1):
    bigram = (tokens[i], tokens[i+1])
    bigram_counts[bigram] = bigram_counts.get(bigram, 0) + 1

# Count unigrams (denominators)
unigram_counts = Counter(tokens)

# Compute probabilities
bigram_probs = {}
for (w1, w2), count in bigram_counts.items():
    bigram_probs[(w1, w2)] = count / unigram_counts[w1]
```

Another Bigram Example (Setup)

Corpus: "the cat sat. the dog sat. the cat ran."

Tokens: [the, cat, sat, the, dog, sat, the, cat, ran]

Goal: Compute selected bigram probabilities

Step 1: Count Bigrams

Bigram	Count
(the, cat)	2
(the, dog)	1
(cat, sat)	1
(cat, ran)	1
(dog, sat)	1
(sat, the)	2

Step 2: Count Unigrams

Word	Count
the	3
cat	2
dog	1
sat	2
ran	1

Step 3: Compute $P(\text{cat} | \text{the})$

$$P(\text{cat} | \text{the}) = \frac{\text{count}(\text{the}, \text{cat})}{\text{count}(\text{the})} = \frac{2}{3} = 0.667$$

Interpretation:

67% of the time "the" appears, it's followed by "cat"

Step 4: Compute $P(\text{dog} - \text{the})$

$$P(\text{dog}|\text{the}) = \frac{\text{count}(\text{the, dog})}{\text{count}(\text{the})} = \frac{1}{3} = 0.333$$

Interpretation:

33% of the time "the" appears, it's followed by "dog"

Step 4: Compute $P(\text{dog} - \text{the})$

$$P(\text{dog}|\text{the}) = \frac{\text{count}(\text{the, dog})}{\text{count}(\text{the})} = \frac{1}{3} = 0.333$$

Interpretation:

33% of the time "the" appears, it's followed by "dog"

Step 5: Verify Distribution

Check: Does $P(\cdot|\text{the})$ sum to 1?

$$P(\text{cat}|\text{the}) + P(\text{dog}|\text{the}) = 0.667 + 0.333 = 1.0 \quad \checkmark$$

Yes! This is a valid conditional distribution.

Step 6: Compute $P(\text{sat} - \text{cat})$

$$P(\text{sat}|\text{cat}) = \frac{\text{count}(\text{cat}, \text{sat})}{\text{count}(\text{cat})} = \frac{1}{2} = 0.5$$

Similarly:

$$P(\text{ran}|\text{cat}) = \frac{\text{count}(\text{cat}, \text{ran})}{\text{count}(\text{cat})} = \frac{1}{2} = 0.5$$

Bigram Model Summary

Key formula (you'll use this constantly):

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$$

Process:

- ① Count all bigrams
- ② Count all unigrams (first words)
- ③ Divide bigram count by unigram count
- ④ Verify each distribution sums to 1

General N-gram Formula

For any n:

$$P(w_n | w_1, \dots, w_{n-1}) = \frac{\text{count}(w_1, \dots, w_n)}{\text{count}(w_1, \dots, w_{n-1})}$$

Pattern:

- Count n-gram in numerator
- Count (n-1)-gram in denominator

Why Smoothing

Problem: Unseen n-grams get probability 0

Training: "I like cats"

Test: "I like dogs"

Never saw (like, dogs) $\rightarrow P(\text{dogs} \mid \text{like}) = 0/1 = 0$

Solution: Add-k smoothing (also called Laplace smoothing)

Add-k Smoothing Formula

Standard MLE (no smoothing):

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$$

Add-k smoothing:

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2) + k}{\text{count}(w_1) + k|V|}$$

where:

- k = smoothing parameter (often 0.1 or 1.0)
- $|V|$ = vocabulary size

Example: Add-k Smoothing (Setup)

Let's compute smoothed bigram probabilities.

Training: "I like cats"

Tokens: [I, like, cats]

Vocabulary: {I, like, cats} $\rightarrow$ $|V| = 3$

Smoothing: $k = 1.0$ (add-1 smoothing)

Goal: Compute $P(\text{dogs} \mid \text{like})$ for unseen word "dogs"

Step 1: Count Bigrams (Training Data)

Observed bigrams:

Bigram	Count
(I, like)	1
(like, cats)	1

Count unigrams:

- $\text{count}(\text{like}) = 1$

Step 2: Standard MLE (No Smoothing)

Without smoothing:

$$P(\text{cats}|\text{like}) = \frac{\text{count}(\text{like}, \text{cats})}{\text{count}(\text{like})} = \frac{1}{1} = 1.0$$

$$P(\text{dogs}|\text{like}) = \frac{\text{count}(\text{like}, \text{dogs})}{\text{count}(\text{like})} = \frac{0}{1} = 0.0$$

Problem: "dogs" impossible! Gets zero probability.

Step 3: Apply Add-1 Smoothing

With $k = 1.0$:

$$P(\text{cats}|\text{like}) = \frac{\text{count}(\text{like}, \text{cats}) + 1}{\text{count}(\text{like}) + 1 \times 3} = \frac{1 + 1}{1 + 3} = \frac{2}{4} = 0.5$$

$$P(\text{dogs}|\text{like}) = \frac{\text{count}(\text{like}, \text{dogs}) + 1}{\text{count}(\text{like}) + 1 \times 3} = \frac{0 + 1}{1 + 3} = \frac{1}{4} = 0.25$$

Step 4: Compute all Smoothed Probabilities

For every word in vocabulary, compute $P(w \mid \text{like})$:

$$\begin{aligned}P(l \mid \text{like}) &= \frac{0 + 1}{1 + 3} = \frac{1}{4} = 0.25 \\P(\text{like} \mid \text{like}) &= \frac{0 + 1}{1 + 3} = \frac{1}{4} = 0.25 \\P(\text{cats} \mid \text{like}) &= \frac{1 + 1}{1 + 3} = \frac{2}{4} = 0.50\end{aligned}$$

Note: "dogs" not in training vocab, but we can still compute:

$$P(\text{dogs} \mid \text{like}) = \frac{0 + 1}{1 + 3} = 0.25$$

Step 5: Verify Distribution Sums to 1

Check: Do probabilities sum to 1?

$$\begin{aligned} &P(I|like) + P(like|like) + P(cats|like) \\ &= 0.25 + 0.25 + 0.50 \\ &= 1.0 \quad \checkmark \end{aligned}$$

Yes! Valid probability distribution.

Note: If we include "dogs" in vocabulary, need to recompute with $|V| = 4$

Effect of Smoothing Parameter k

Try different k values for $P(\text{cats} \mid \text{like})$:

k = 0 (no smoothing):

$$P(\text{cats} \mid \text{like}) = \frac{1 + 0}{1 + 0} = \frac{1}{1} = 1.0$$

k = 0.1 (light smoothing):

$$P(\text{cats} \mid \text{like}) = \frac{1 + 0.1}{1 + 0.3} = \frac{1.1}{1.3} = 0.846$$

k = 1.0 (add-1):

$$P(\text{cats} \mid \text{like}) = \frac{1 + 1}{1 + 3} = \frac{2}{4} = 0.5$$

Smoothing Tradeoffs

Small k (e.g., 0.1):

- Stays close to observed data
- Unseen n -grams get low probability
- Less diversity in generation

Large k (e.g., 1.0):

- More probability mass to unseen n -grams
- Observed data counts less
- More diversity but less accuracy

Smoothing in Python

```
def bigram_prob_smoothed(w1, w2, counts, unigrams,
                        vocab_size, k=1.0):
    """Compute  $P(w2|w1)$  with add-k smoothing."""
    numerator = counts.get((w1, w2), 0) + k
    denominator = unigrams.get(w1, 0) + k * vocab_size

    if denominator == 0:
        return 1.0 / vocab_size # Uniform

    return numerator / denominator

# Example
V = 3 # vocab size
p = bigram_prob_smoothed("like", "cats",
                        bigram_counts,
                        unigram_counts, V, k=1.0)
print(f"P(cats|like) = {p}") # 0.5
```



Zero Probability Problem

What if we never saw a bigram?

Training: "the cat sat on the mat"

Test: "the dog sat"

We never saw "the dog" $\rightarrow$ count = 0

$$P(\text{dog}|\text{the}) = \frac{0}{2} = 0$$

Entire sentence gets probability 0!

Sparsity Gets Worse w/ Higher N

As n increases:

- Unigrams: V possible
- Bigrams: V^2 possible
- Trigrams: V^3 possible

Most n-grams never appear in training!

This motivates smoothing (next lecture)

Text Generation

Use n-gram model to generate text:

- 1 Start with context
- 2 Sample next word from $P(\cdot \text{ — context})$
- 3 Update context with sampled word
- 4 Repeat until done

Generate Code

```
import random

def generate(model, start, length=10):
    """Generate text using bigram model."""
    current = start
    result = [current]

    for _ in range(length):
        # Get distribution  $P(/current)$ 
        candidates = {}
        for (w1, w2), prob in model.items():
            if w1 == current:
                candidates[w2] = prob

        if not candidates:
            break
```

Generate Code (cont.)

```
# Sample next word
words = list(candidates.keys())
probs = list(candidates.values())
next_word = random.choices(words,
                            weights=probs)[0]

result.append(next_word)
current = next_word

return " ".join(result)
```

Prediction

Predict next k words by sliding window:

For trigram model:

- Use last 2 words as context
- Predict word 1, add to sequence
- Slide window (now use predicted + previous)
- Predict word 2, repeat k times

Why Language Models Matter

Applications:

- Speech recognition
- Machine translation
- Text generation
- Autocomplete
- Spell correction

Core idea: Model what sequences are "likely"

Key Formulas (memorize these)

Unigram:

$$P(w) = \frac{\text{count}(w)}{N}$$

Bigram:

$$P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$$

Trigram:

$$P(w_3|w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$$

Add-k smoothing (any n-gram):

$$P(w_n|\text{context}) = \frac{\text{count}(\text{context}, w_n) + k}{\text{count}(\text{context}) + k|V|}$$

◁ ○ ▷ ↻ 🔍



Thank you!

Date